DSC 210 Numerical Linear Algebra, Fall 2025

Homework Problems for Topic 3: *Eigenvalue Problems*

Student Name (PID):

Write your solutions to the following problems by typing them in LaTeX. Unless otherwise noted by the problem's instructions, show your work and provide justification for your answer. Homework is due via Gradescope at **25th November 2025, 11:59 PM**.
Late Policy: If you submit your homework after the deadline we will apply a late penalty of 10% per day.

Guidelines for Homework Related Questions:

(a) As a general rule, we can help you understand the homework problems and explain the material from the corresponding lectures, but we cannot give you the entire solution.

(b) Regarding debugging programming questions: We ask you to do some debugging on your own first, including printing out intermediate values in your algorithms, trying a simpler version of the problem, etc.

(c) We will not be pre-grading the homework, i.e. we won't confirm if the answer you have is correct.

AI Usage Policy:

(a) Code: You may use LLMs to debug your code; however, you may not use LLMs to generate your entire code, and code must be reviewed and tested.

(b) Writing: You may use LLMs to correct grammar, style and latex issues; however, you may not use LLMs to generate entire solutions, sentences or paragraphs. All writing must be in your own voice.

Academic Integrity Policy:

The UC San Diego Academic Integrity Policy (formerly the Policy on Integrity of Scholarship) is effective as of September 25, 2023 and applies to any cases originating on or after September 25, 2023. The university expects both faculty and students to honor the policy. For students, this means that all academic work will be done by the individual to whom it's assigned, without unauthorized aid of any kind. If violations of academic integrity occur, the same Sanctioning Guidelines apply regardless of which policy was effective for that case.

For more information on how the policy is implemented, refer to the most current procedures. Remember: When in doubt about what constitutes appropriate collaboration or resource use, please ask TAs before proceeding. It's always better to clarify expectations than to risk an academic integrity violation. Academic integrity violations can have serious consequences for your academic record, and you will get zero grades.

You can access the Homework Template using the following link: `https://www.overleaf.com/read/vfhcmsppvskp`

Question 1: Singular value decomposition (25 points)
Determine the SVD of the following matrices by hand calculation. Make sure to explicitly list Σ, U, and V in your answer.

(a) $\begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$ (4 points)

(b) $\begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}$ (4 points)

(c) $\begin{bmatrix} 0 & 3 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$ (4 points)

(d) $\begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$ (4 points)

(e) $\begin{bmatrix} 2 & 2 \\ 3 & 2 \end{bmatrix}$ (4 points)

(f) Write Python code to compute SVD of the matrices in parts (a) to (e) and verify that the above results match what you got from code. (5 points)
Hint: You can use the numpy library function.

Solution:

..... Exercise 1A .....

$$\mathbf{A} = \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix} \qquad \mathbf{A}^\top = \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix} \qquad \mathbf{A}\mathbf{A}^\top = \begin{bmatrix} 25 & 0 \\ 0 & 4 \end{bmatrix}$$

$$\det\left(AA^\top - \lambda I\right) = 0$$
$$\det \begin{bmatrix} 25-\lambda & 0 \\ 0 & 4-\lambda \end{bmatrix} = 0$$

$(25-\lambda)(4-\lambda) = 0)$

Eigenvalues: $\lambda_1 = 25, \lambda_2 = 4$

Singular Values: $\sigma_1 = 5, \sigma_2 = 2$

$$\Sigma = \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$$

Calculate eignvectors of $AA^\top$:

Eigenvector for $\lambda_1 = 25$

$$\begin{bmatrix} 25-25 & 0 \\ 0 & 4-25 \end{bmatrix} \begin{bmatrix} u_{1x} \\ u_{1y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & 0 \\ 0 & -21 \end{bmatrix} \begin{bmatrix} u_{1x} \\ u_{1y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$u_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Eigenvector for $\lambda_2 = 4$

$$\begin{bmatrix} 25-4 & 0 \\ 0 & 4-4 \end{bmatrix} \begin{bmatrix} u_{2x} \\ u_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 21 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_{2x} \\ u_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$u_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$U = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Calculate eignvectors of $A^\top A$:

Eigenvector for $\lambda_1 = 25$

$$\begin{bmatrix} 25-25 & 0 \\ 0 & 4-25 \end{bmatrix} \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & 0 \\ 0 & -21 \end{bmatrix} \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Eigenvector for $\lambda_2 = 4$

$$\begin{bmatrix} 25-4 & 0 \\ 0 & 4-4 \end{bmatrix} \begin{bmatrix} v_{2x} \\ v_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 21 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_{2x} \\ v_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$A = U\Sigma V^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Exercise 1B

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix} \qquad \mathbf{A}^\top = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix} \qquad \mathbf{A}\mathbf{A}^\top = \begin{bmatrix} 4 & 0 \\ 0 & 25 \end{bmatrix}$$

$$\det\left(AA^\top - \lambda I\right) = 0 \det \begin{bmatrix} 4-\lambda & 0 \\ 0 & 25-\lambda \end{bmatrix} \qquad = 0$$

$(4-\lambda)(25-\lambda)=0$

Eigenvalues: $\lambda_1=4, \lambda_2=25$

Singular Values: $\sigma_1=2, \sigma_2=5$

$$\Sigma=\begin{bmatrix}5 & 0\\ 0 & 2\end{bmatrix}$$

Calculate eignvectors of $AA^\top$:

Eigenvector for $\lambda_1=4$

$$\begin{bmatrix}4-4 & 0\\ 0 & 25-4\end{bmatrix}\begin{bmatrix}u_{1x}\\ u_{1y}\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$$
$$\begin{bmatrix}0 & 0\\ 0 & 21\end{bmatrix}\begin{bmatrix}u_{1x}\\ u_{1y}\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$$
$$u_1=\begin{bmatrix}0\\ 1\end{bmatrix}$$

Eigenvector for $\lambda_2=25$

$$\begin{bmatrix}4-25 & 0\\ 0 & 25-25\end{bmatrix}\begin{bmatrix}u_{2x}\\ u_{2y}\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$$
$$\begin{bmatrix}-21 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}u_{2x}\\ u_{2y}\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$$
$$u_2=\begin{bmatrix}1\\ 0\end{bmatrix}$$

$$U=\begin{bmatrix}0 & 1\\ 1 & 0\end{bmatrix}$$

Calculate eignvectors of $A^\top A$:

Eigenvector for $\lambda_1=4$

$$\begin{bmatrix}4-4 & 0\\ 0 & 25-4\end{bmatrix}\begin{bmatrix}v_{1x}\\ v_{1y}\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$$
$$\begin{bmatrix}0 & 0\\ 0 & 21\end{bmatrix}\begin{bmatrix}v_{1x}\\ v_{1y}\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$$
$$v_1=\begin{bmatrix}0\\ 1\end{bmatrix}$$

Eigenvector for $\lambda_2=25$

$$\begin{bmatrix}4-25 & 0\\ 0 & 25-25\end{bmatrix}\begin{bmatrix}v_{2x}\\ v_{2y}\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$$
$$\begin{bmatrix}-21 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}v_{2x}\\ v_{2y}\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$$
$$v_2=\begin{bmatrix}1\\ 0\end{bmatrix}$$

$$V = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$A = U\Sigma V^\top = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

..... Exercise 1C .....

$$\mathbf{A} = \begin{bmatrix} 0 & 3 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \qquad \mathbf{A}^\top = \begin{bmatrix} 0 & 0 & 0 \\ 3 & 0 & 0 \end{bmatrix} \qquad \mathbf{A}\mathbf{A}^\top = \begin{bmatrix} 9 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\det\left(AA^\top - \lambda I\right) = 0$$
$$\det \begin{bmatrix} 9-\lambda & 0 & 0 \\ 0 & 0-\lambda & 0 \\ 0 & 0 & 0-\lambda \end{bmatrix} = 0$$

$(9-\lambda)(0-\lambda)(0-\lambda) = 0$

Eigenvalues: $\lambda_1 = 9, \lambda_2 = 0$

Singular Values: $\sigma_1 = 3, \sigma_2 = \sigma_3 = 0$

$$\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Calculate eignvectors of $AA^\top$:

Eigenvector for $\lambda_1 = 3$

$$\begin{bmatrix} 9-3 & 0 & 0 \\ 0 & 0-3 & 0 \\ 0 & 0 & 0-3 \end{bmatrix} \begin{bmatrix} u_{1x} \\ u_{1y} \\ u1_z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 6 & 0 & 0 \\ 0 & -3 & 0 \\ 0 & 0 & -3 \end{bmatrix} \begin{bmatrix} u_{1x} \\ u_{1y} \\ u1_z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$
$$u_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Eigenvector for $\lambda_2 = 9$

$$\begin{bmatrix} 9-9 & 0 & 0 \\ 0 & 0-9 & 0 \\ 0 & 0 & 0-9 \end{bmatrix} \begin{bmatrix} u_{2x} \\ u_{2y} \\ u2_z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & -9 & 0 \\ 0 & 0 & -9 \end{bmatrix} \begin{bmatrix} u_{2x} \\ u_{2y} \\ u2_z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$u_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Eigenvector for $\lambda_2 = 0$

$$\begin{bmatrix} 0-3 & 0 & 0 \\ 0 & 0-0 & 0 \\ 0 & 0 & 0-0 \end{bmatrix} \begin{bmatrix} u_{2x} \\ u_{2y} \\ u2_z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} -3 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u2x \\ u_{2y} \\ u2_z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$u = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

Due to degenerate eigenvalues,

$$u_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \; u_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$U = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Calculate eignvectors of $A^\top A$:

Eigenvector for $\lambda_1 = 9$

$$\begin{bmatrix} 0-9 & 0 \\ 0 & 9-9 \end{bmatrix} \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} -9 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$v_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Eigenvector for $\lambda_2 = 0$

$$\begin{bmatrix} 0-0 & 0 \\ 0 & 9-0 \end{bmatrix} \begin{bmatrix} v_{2x} \\ v_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & 0 \\ 0 & 9 \end{bmatrix} \begin{bmatrix} v_{2x} \\ v_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$V = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$A = U\Sigma V^\top = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

..... Exercise 1D .....

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \qquad \mathbf{A}^\top = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \qquad \mathbf{A}\mathbf{A}^\top = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\det\left(AA^\top - \lambda I\right) = 0$$
$$\det \begin{bmatrix} 1-\lambda & 1 \\ 1 & 1-\lambda \end{bmatrix} = 0$$

$(1-\lambda)(1-\lambda) - (1)(1) = 0$
Eigenvalues: $\lambda_1 = 2, \lambda_2 = 0$
Singular Values: $\sigma_1 = \sqrt{2}, \sigma_2 = 0$

$$\Sigma = \begin{bmatrix} \sqrt{2} & 0 \\ 0 & 0 \end{bmatrix}$$

Calculate eignvectors of $AA^\top$:
Eigenvector for $\lambda_1 = 2$

$$\begin{bmatrix} 1-2 & 1 \\ 1 & 1-2 \end{bmatrix} \begin{bmatrix} u_{1x} \\ u_{1y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$u_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$$

Eigenvector for $\lambda_2 = 0$

$$\begin{bmatrix} 1-0 & 1 \\ 1 & 1-0 \end{bmatrix} \begin{bmatrix} u_{2x} \\ u_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$u_2 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{bmatrix}$$

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}$$

Calculate eignvectors of $A^\top A$:

Eigenvector for $\lambda_1 = 2$

$$\begin{bmatrix} 2-2 & 0 \\ 0 & 0-2 \end{bmatrix} \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Eigenvector for $\lambda_2 = 0$

$$\begin{bmatrix} 2-0 & 0 \\ 0 & 0-0 \end{bmatrix} \begin{bmatrix} v_{2x} \\ v_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$A = U\Sigma V^\top = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \sqrt{2} & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

..... Exercise 1E .....

$$\mathbf{A} = \begin{bmatrix} 2 & 2 \\ 3 & 2 \end{bmatrix} \qquad \mathbf{A}^\top = \begin{bmatrix} 2 & 3 \\ 2 & 2 \end{bmatrix} \qquad \mathbf{A}\mathbf{A}^\top = \begin{bmatrix} 8 & 10 \\ 10 & 13 \end{bmatrix}$$

$$\det\left(AA^\top - \lambda I\right) = 0$$
$$\det \begin{bmatrix} 8-\lambda & 10 \\ 10 & 13-\lambda \end{bmatrix} = 0$$

$(8-\lambda)(13-\lambda)-(10)(10)=0$

Eigenvalues: $\lambda_1=\frac{21+5\sqrt{17}}{2}, \lambda_2=\frac{21-5\sqrt{17}}{2}$

Singular Values: $\sigma_1=4.56155281280883, \sigma_2=0.4384471871911691$

$$\Sigma=\begin{bmatrix}4.562 & 0\\ 0 & 0.438\end{bmatrix}$$

Calculate eignvectors of $AA^\top$:

Eigenvector for $\lambda_1=\frac{21+5\sqrt{17}}{2}$

$$\begin{bmatrix}8-\frac{21+5\sqrt{17}}{2} & 10\\ 10 & 13-\frac{21+5\sqrt{17}}{2}\end{bmatrix}\begin{bmatrix}u_{1x}\\ u_{1y}\end{bmatrix}=\begin{bmatrix}0\\0\end{bmatrix}$$

$$u_1=\begin{bmatrix}-0.615\\-0.788\end{bmatrix}$$

Eigenvector for $\lambda_2=\frac{21-5\sqrt{17}}{2}$

$$\begin{bmatrix}8-\frac{21-5\sqrt{17}}{2} & 10\\ 10 & 13-\frac{21-5\sqrt{17}}{2}\end{bmatrix}\begin{bmatrix}u_{1x}\\ u_{1y}\end{bmatrix}=\begin{bmatrix}0\\0\end{bmatrix}$$

$$u_2=\begin{bmatrix}-0.788\\ 0.615\end{bmatrix}$$

$$U=\begin{bmatrix}-0.615 & -0.788\\ -0.788 & 0.615\end{bmatrix}$$

Calculate eignvectors of $A^\top A$:

Eigenvector for $\lambda_1=\frac{21+5\sqrt{17}}{2}$

$$\begin{bmatrix}13-\frac{21+5\sqrt{17}}{2} & 10\\ 10 & 8-\frac{21+5\sqrt{17}}{2}\end{bmatrix}\begin{bmatrix}v_{1x}\\ v_{1y}\end{bmatrix}=\begin{bmatrix}0\\0\end{bmatrix}$$

$$v_1=\begin{bmatrix}-0.788\\-0.615\end{bmatrix}$$

Eigenvector for $\lambda_2=\frac{21-5\sqrt{17}}{2}$

$$\begin{bmatrix}13-\frac{21-5\sqrt{17}}{2} & 10\\ 10 & 8-\frac{21-5\sqrt{17}}{2}\end{bmatrix}\begin{bmatrix}v_{2x}\\ v_{2y}\end{bmatrix}=\begin{bmatrix}0\\0\end{bmatrix}$$

$$v_2=\begin{bmatrix}0.615\\-0.788\end{bmatrix}$$

$$V = \begin{bmatrix} -0.788 & -0.615 \\ 0.615 & -0.788 \end{bmatrix}$$

$$A = U\Sigma V^\top = \begin{bmatrix} -0.615 & -0.788 \\ -0.788 & 0.615 \end{bmatrix} \begin{bmatrix} 4.561 & 0 \\ 0 & 0.438 \end{bmatrix} \begin{bmatrix} -0.788 & -0.615 \\ 0.615 & -0.788 \end{bmatrix}$$

Exercise 1F: Python Implementation

```python
import numpy as np

matrix_a = [[5, 0], [0, 2]]
matrix_b = [[2, 0], [0, 5]]
matrix_c = [[0, 3], [0, 0], [0, 0]]
matrix_d = [[1, 0], [1, 0]]
matrix_e = [[2, 2], [3, 2]]

svd_a = np.linalg.svd(matrix_a)
print(f'svd_a: {svd_a}')

svd_b = np.linalg.svd(matrix_b)
print(f'svd_b: {svd_b}')

svd_c = np.linalg.svd(matrix_c)
print(f'svd_c: {svd_c}')

svd_d = np.linalg.svd(matrix_d)
print(f'svd_d: {svd_d}')

svd_e = np.linalg.svd(matrix_e)
print(f'svd_e: {svd_e}')
```

Console Outputs

```
svd_a: SVDResult(U=array([[1., 0.],
       [0., 1.]]), S=array([5., 2.]), Vh=array([[1., 0.],
       [0., 1.]]))

svd_b: SVDResult(U=array([[0., 1.],
       [1., 0.]]), S=array([5., 2.]), Vh=array([[0., 1.],
       [1., 0.]]))

svd_c: SVDResult(U=array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]]), S=array([3., 0.]), Vh=array([[ 0.,  1.],
       [-1.,  0.]]))

svd_d: SVDResult(U=array([[-0.70710678, -0.70710678],
       [-0.70710678,  0.70710678]]), S=array([1.41421356, 0.        ]), Vh=array
    ([[-1., -0.],
       [ 0.,  1.]]))

svd_e: SVDResult(U=array([[-0.61541221, -0.78820544],
       [-0.78820544,  0.61541221]]), S=array([4.56155281, 0.43844719]), Vh=array
    ([[-0.78820544, -0.61541221],
       [ 0.61541221, -0.78820544]]))
```

Question 2: Eigenvalues and Eigenvectors (25 points)

$$\mathbf{A} = \begin{bmatrix} -5 & 3 \\ -6 & 6 \end{bmatrix}$$

(a) Write down the characteristic equation for matrix **A**. Use the characteristic equation to solve the eigenvalues and normalized eigenvectors of matrix **A**. (10 points)

(b) Write python code to verify your answers for the eigenvalues and normalized eigenvectors from Part (a). **Hint**: You can use numpy to solve this problem. (5 points)

(c) Prove that if a real matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ has unique eigenvalues, then the eigenvectors $\mathbf{x}_i$ are linearly independent. **Hint:** Prove by contradiction, start with $n = 2$ case. (10 points)

Solution:

..... Exercise 2A .....

The characteristic equation is

$$\det(A - \lambda I) = 0$$
$$\det \begin{bmatrix} -5-\lambda & 3 \\ -6 & 6-\lambda \end{bmatrix} = 0$$

$$(-5-\lambda)(6-\lambda) - (3)(-6) = 0$$
$$(-30 + 5\lambda - 6\lambda - \lambda^2 + 18 = 0$$
$$(\lambda + 4)(\lambda - 3) = 0$$

The eigenvalues are

$\lambda_1 = 4, \lambda_2 = -3$

The normalized eigenvectors are

For $\lambda_1 = 4$

$$\begin{bmatrix} -5-(4) & 3 \\ -6 & 6-(4) \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 0$$
$$\begin{bmatrix} -9 & 3 \\ -6 & 2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 0$$

Therefore,

$$x_2 = 3x_1$$

The norm is:

$$\|v\| = \sqrt{1^2 + 3^2}$$
$$= \sqrt{10}$$

The normalized eigenvector is

$$u = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

For $\lambda_2 = -3$

$$\begin{bmatrix} -5-(-3) & 3 \\ -6 & 6-(-3) \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 0$$
$$\begin{bmatrix} -2 & 3 \\ -6 & 9 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 0$$

Therefore,

$$x_2 = \frac{3}{2} x_1$$

The norm is:

$$\begin{aligned} \|v\| &= \sqrt{1^2 + \frac{2}{3}^2} \\ &= \frac{\sqrt{13}}{3} \quad \text{But, since eigenvectors can have scalars,} \\ &= \frac{\sqrt{13}}{3} * 3 \\ &= \sqrt{13} \end{aligned}$$

The normalized eigenvector is

$$u = \frac{1}{\sqrt{13}} \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Exercise 2B

```
matrix_A = [[−5, 3], [−6, 6]]
eigenvalues, eigenvectors = np.linalg.eig(matrix_A)
print(f'eigenvalues: {eigenvalues}, \neigenvectors: {eigenvectors}')
print(f'eigenvector norm: {np.linalg.norm(eigenvectors[:, 0])}')
print(f'eigenvector norm: {np.linalg.norm(eigenvectors[:, 1])}')
```

```
eigenvalues: [−3.  4.],
eigenvectors: [[−0.83205029 −0.31622777]
 [−0.5547002  −0.9486833 ]]
eigenvector norm: 1.0
eigenvector norm: 0.9999999999999999
```

Exercise 2C

Assume $A \in \mathbb{R}^{2x2}$ has unique eigenvalues λ_1, λ_2. Let $Ax_1 = \lambda_1 x_1$ and $Ax_2 = \lambda_2 x_2$. Suppose that with the starting case of $n = 2$, that is to say two eigenvectors x_1, x_2 are linearly independent.

Now, let there exist a scalar c such that $x_2 = cx_1$ with c $\in \mathbb{R}$. This means that it can be substituted into the second equation such that:

$$\begin{aligned} Ax_2 &= \lambda_2 x_2 \\ A(cx_1) &= \lambda_2(cx_1) \\ c(Ax_1) &= c(\lambda_2 x_1) \\ (Ax_1) &= (\lambda_2 x_1) \end{aligned}$$

But this is a contradiction. The assumption has stated that by the definition of eigenvectors, $Ax_1 = \lambda_1 x_1$. Also, because $A = \lambda_1$ further substition demonstrates the following contradiction:

$$\begin{aligned} \lambda_1 x_1 &= \lambda_2 x_2 \\ x_1 &= x_2 \end{aligned}$$

which is nonsense if the eigenvectors are linearly independent (one is not a scalar of another). Therefore x_1 and x_2 are NOT linearly independent. A similar argument can be made for any case with $n > 2$, for any scalar c $\in \mathbb{R}$.

Question 3: Power method (20 points)

Solve the problem in the google colab. Please make a copy of the notebook and solve it there.

(a) Power Method: Write function `power_method(A,x)`, which takes as input matrix A and a vector $\mathbf{x}$, and uses the power method to calculate eigenvalues and eigenvectors.
Get the largest **(in absolute value)** eigenvalue and the corresponding eigenvector for matrix A using the above function.

$$A = \begin{bmatrix} 2 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 4 & 1 \end{bmatrix}$$

Start with initial eigenvector guesses: $[-1, 0.5, 3]$ and $[2, -6, 0.2]$. For each of the vectors, iterate until convergence.

i. Plot how the eigenvalue changes with respect to iterations.
ii. Report the number of steps it took to converge, for both the eigenvalue and eigenvector.
iii. Report the final eigenvalue and eigenvector. Match your output with the results generated by the numpy API: `numpy.linalg.eig`.

Note: You only need to look at magnitudes of eigenvalues. Use an absolute tolerance of 10^{-6} between eigenvalue output of previous and current iteration as stopping criteria. You may also need to normalize the final eigenvector to match with output of numpy API `numpy.linalg.eig`. (10 points)

(b) Inverse Power Method:
Write a function `inverse_power_method(A,x)`, which takes as input matrix $\mathbf{A}$ and a vector $\mathbf{x}$, and uses inverse power method to calculate the smallest (in absolute value) eigenvalue and corresponding eigenvector. Solve for the smallest (in absolute value) eigenvalue and corresponding eigenvector for the matrix from (a). Use the same initial eigenvector guesses as (a).

i. Plot the computed/estimated eigenvalue with respect to iterations.
ii. Report how many iterations do you need for it to converge to the smallest eigenvalue.
iii. Report the final eigenvalue and eigenvector you get. Match your answer with the results generated by the numpy API `numpy.linalg.eig`.

Note: You only need to look at magnitudes of eigenvalues. Use an absolute tolerance of 10^{-6} between eigenvalue output of previous and current iteration as stopping criteria. You may also need to normalize the final eigenvector to match with output of numpy API `numpy.linalg.eig`. (10 points)

Solution:

Question 4: Face Recognition with Eigenfaces (30 points + 10 Bonus Points)

Solve the problem in the google colab. Please make a copy of the notebook and solve it there.

Goal: Perform face recognition on the *Labeled Faces in the Wild* dataset using PyTorch.

Dataset Information: Labeled Faces in the Wild dataset consists of face photographs designed for studying the problem of unconstrained face recognition. The original dataset contains more than 13,000 images of faces collected from the web.

Tasks:

- First, perform Principal Component Analysis (PCA) on the image dataset.
- Using PCA, extract the Top k principal components (eigenvalues).
- Reconstruction of faces from these **eigenvalues** will give us the **eigen-faces** which are the most representative features of most of the images in the dataset.
- **BONUS**: Finally, train a simple PyTorch Neural Network model on the modified image dataset. This trained model will be used for prediction and evaluation on a test set.

Problem Description

(a) Preprocessing: Using the `train_test_split` API from `sklearn`, split the data into train and test dataset in the ratio 3:1. Use `random_state=42`.

For better performance, normalize the features which can have different ranges with huge values. (As all our features here are in the range [0, 255], it is not explicitly needed here. However, it is a good exercise.)

Use the `StandardScaler` class from `sklearn` and use that to normalize `X_train` and `X_test`. Validate and show your result by printing the first 5 features of 5 images of `X_train` (This result can vary from PC to PC). (10 points)

(b) Dimensionality reduction: Use the PCA API from `sklearn` to extract the top 100 principal components of the image matrix and fit it on the training dataset.

Visualize some of the top few components as an image (eigenfaces). (10 points)

(c) Face reconstruction: Reconstruct an image from its point projected on the principal component basis. Project the first three faces on the eigenvector basis using PCA models trained with varying number of principal components. Using the projected points, reconstruct the faces, and visualize the images.

Your final output should be a (3×5) image matrix, where the rows are the data points, and the columns correspond to original image and reconstructed image for n_components = $[10, 100, 150, 500]$. (10 points)

(d) Prediction **(BONUS)**: Train a neural network classifier in **PyTorch** on the transformed dataset. Note: For PyTorch reference see documentation (10 points)

Solution:

Homework 3

This notebook contains all code for Homework 3 of DSC 210.

- Question 3: Power method (20 points)
- Question 4: Face Recognition with Eigenfaces (30 points + 10 bonus points)

Notes:

- For programming solutions, properly add comments to your code.

Question 3: Power method (20 points)

Part (a): `power_method(A, x)`

Write function `power_method(A,x)`, which takes as input matrix A and a vector $\mathbf{x}$, and uses the power method to calculate eigenvalues and eigenvectors.

Get the largest **(in absolute value)** eigenvalue and the corresponding eigenvector for matrix A using the above function.

$$A = \begin{bmatrix} 2 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 4 & 1 \end{bmatrix}$$

Start with intial eigenvector guesses: `[-1, 0.5, 3]` and `[2,-6,0.2]`. For each of the vectors, iterate until convergence.

(i) Plot how the eigenvalue changes w.r.t. iterations.

(ii) Report the number of steps it took to converge, for both the eigenvalue and eigenvector.

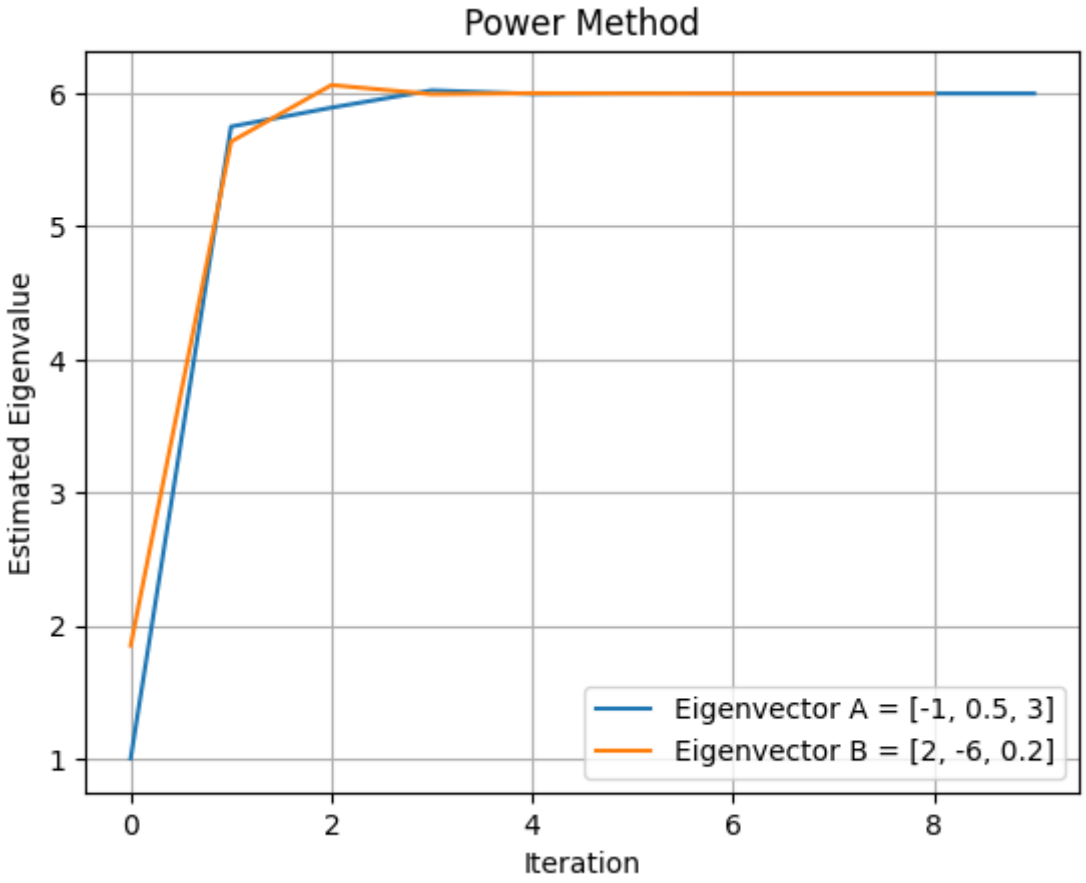
(iii) Report the final eigenvalue and eigenvector. Match your output with the results generated by the numpy API: `numpy.linalg.eig`

Note: You only need to look at magnitudes of eigenvalues. Use an absolute tolerance of 10^{-6} between eigenvalue output of previous and current iteration as stopping criteria. You may also need to normalize the final eigenvector to match with output of numpy API `numpy.linalg.eig`. (10 points)

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

```python
A = np.array([[2, 2, 1], [1, 3, 2], [2, 4, 1]])
# !!!! YOUR CODE HERE !!!!
def power_method(A, x, tolerance=1e-6):
    v = x / np.linalg.norm(x) # initalize non-zero random vector
    lambda_history = []
    steps = 0


    while True: # start iteration
        steps += 1

        w = A @ v # A is a matrix!
        v_update = w / np.linalg.norm(w)

        # current eigenvalue
        rayleigh = v.T @ A @ v # no need to include divide, ||v|| = 1
        lambda_history.append(rayleigh)

        if np.linalg.norm(v_update - v) <= tolerance: # stop criteria from lecture
            if v_update[-1] < 0:
                v_update = -v_update
            return rayleigh, v_update, np.array(lambda_history), steps

        v = v_update

eigvec_a = np.array([-1, 0.5, 3])
eigvec_b = np.array([2, -6, 0.2])

a_lambda, a_v_final, a_lambda_history, a_steps = power_method(A, eigvec_a)
print(f'a_lambda: {a_lambda}; \nv_final: {a_v_final}; \nlambda_history: {np.array(a
print('\n')

b_lambda, b_v_final, b_lambda_history, b_steps = power_method(A, eigvec_b)
print(f'b_lambda: {b_lambda}; \nv_final: {b_v_final}; \nlambda_history: {np.array(b
print('\n')

eigval, eigvec = np.linalg.eig(A)
print(f'Eigenvalues: {eigval}, \nEigenvectors: {eigvec}')

plt.plot(a_lambda_history, label='Eigenvector A = [-1, 0.5, 3]')
plt.plot(b_lambda_history, label='Eigenvector B = [2, -6, 0.2]')
plt.title('Power Method')
plt.xlabel('Iteration')
plt.ylabel('Estimated Eigenvalue')
plt.legend()
plt.grid(True)
plt.show()
```

```
a_lambda: 6.000000515918746;
v_final: [0.46156631 0.59344242 0.6593805 ];
lambda_history: [1.         5.75113122 5.8936644  6.023398   5.99753913 6.00066812
 5.99993204 6.00001857 5.99999811 6.00000052];
steps to convergence: 10


b_lambda: 6.000001525411784;
v_final: [0.46156623 0.59344243 0.65938054];
lambda_history: [1.84915085 5.63805584 6.06342561 5.99509241 6.00197117 5.99986808
 6.00005491 5.99999634 6.00000153];
steps to convergence: 9


Eigenvalues: [ 6.  1. -1.],
Eigenvectors: [[ 4.61566331e-01  8.94427191e-01  5.18104078e-17]
 [ 5.93442426e-01 -4.47213595e-01 -4.47213595e-01]
 [ 6.59380473e-01  3.01974496e-16  8.94427191e-01]]
```



Part (b): `inverse_power_method(A, x)`

Write function `inverse_power_method(A,x)`, which takes as input matrix A and a vector $\mathbf{x}$, and uses inverse power method to calculate the smallest (in absolute value) eigenvalue and corresponding eigenvector. Solve for the smallest (in absolute value) eigenvalue and corresponding eigenvector for the matrix from (a). Use the same intial eigenvector guesses

as (a).

(i) Plot the computed/estimated eigenvalue with respect to iterations.

(ii) Report how many iterations do you need for it to converge to the smallest eigenvalue.

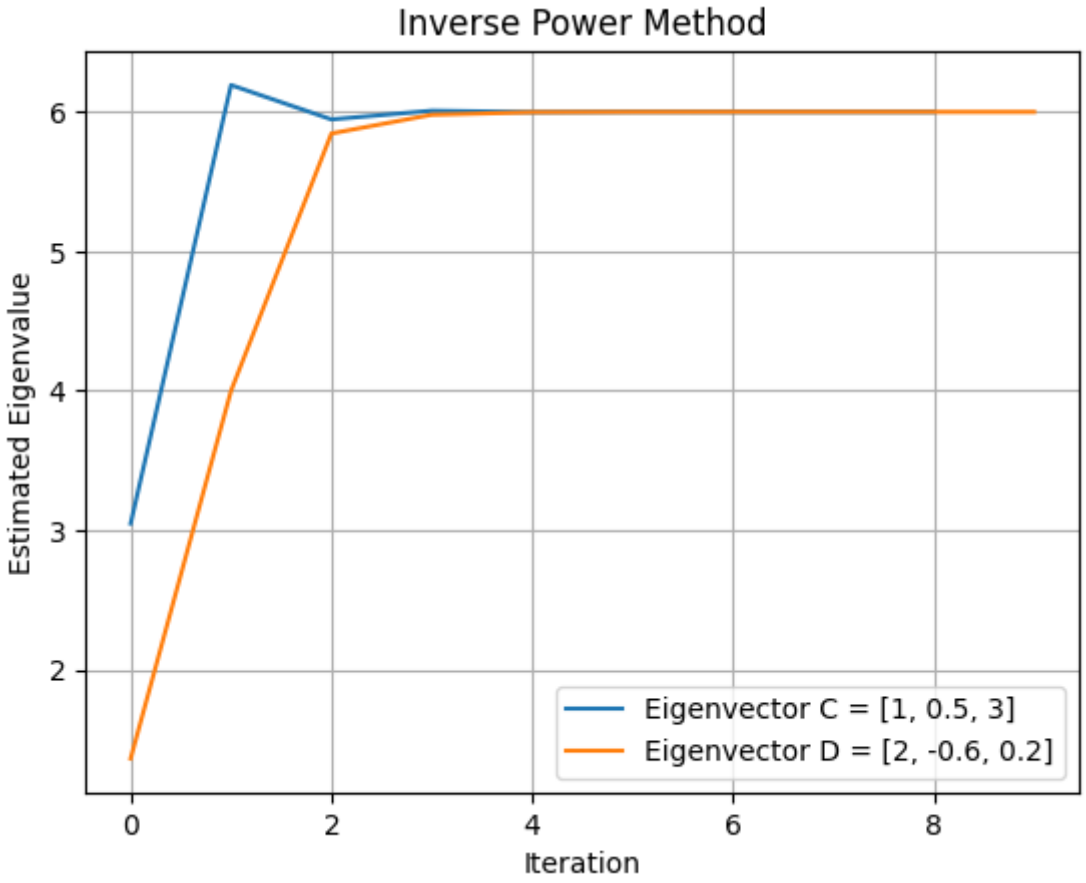
(iii) Report the final eigenvalue and eigenvector you get. Match your answer with the results generated by the **numpy** API `numpy.linalg.eig`.

Note: You only need to look at magnitudes of eigenvalues. Use an absolute tolerance of 10^{-6} between eigenvalue output of previous and current iteration as stopping criteria. You may also need to normalize the final eigenvector to match with output of numpy API `numpy.linalg.eig`. (10 points)

In [2]:

```
# !!!! YOUR CODE HERE !!!!
def inverse_power_method(A, x, tolerance=1e-6):
    v = x / np.linalg.norm(x)
    lambda_history = []
    steps = 0
    sigma = 0 # make this as small as possible
    B = (A - sigma * np.eye(A.shape[0])) # np.eye() size = rows in A

    # basically the same as power method, but eplace A with A - sigma * I
    while True:
        steps += 1

        w = np.linalg.solve(B, v) # (A - sigma * I) w = v
        v_update = w / np.linalg.norm(w)

        # current eigenvalue
        rayleigh = v.T @ B @ v # no need to include divide, ||v|| = 1
        lambda_history.append(rayleigh)

        if np.linalg.norm(v_update - v) <= tolerance: # stop criteria from lecture
            if v_update[-1] < 0:
                v_update = -v_update
            return rayleigh, v_update, np.array(lambda_history), steps

        v = v_update

eigvec_c = np.array([1, 0.5, 3])
eigvec_d = np.array([2, -0.6, 0.2])

c_lambda, c_v_final, c_lambda_history, c_steps = power_method(A, eigvec_c)
print(f'c_lambda: {c_lambda}; \nv_final: {c_v_final}; \nlambda_history: {np.array(c
print('\n')

d_lambda, d_v_final, d_lambda_history, d_steps = power_method(A, eigvec_d)
print(f'd_lambda: {d_lambda}; \nv_final: {d_v_final}; \nlambda_history: {np.array(d
print('\n')

eigval, eigvec = np.linalg.eig(A)
print(f'Eigenvalues: {eigval}, \nEigenvectors: {eigvec}')

plt.plot(c_lambda_history, label='Eigenvector C = [1, 0.5, 3]')
plt.plot(d_lambda_history, label='Eigenvector D = [2, -0.6, 0.2]')
plt.title('Inverse Power Method')
plt.xlabel('Iteration')
plt.ylabel('Estimated Eigenvalue')
plt.legend()
plt.grid(True)
plt.show()
```

```
c_lambda: 5.999998903733644;
v_final: [0.46156634 0.59344248 0.65938042];
lambda_history: [3.04878049 6.19236884 5.94611554 6.00844635 5.99857708 6.00023673
 5.99996053 6.00000658 5.9999989 ];
steps to convergence: 9


d_lambda: 5.999999610417169;
v_final: [0.4615664  0.59344239 0.65938046];
lambda_history: [1.36363636 4.         5.84507042 5.97960432 5.99768484 5.99949337
 5.99993733 5.99998597 5.99999826 5.99999961];
steps to convergence: 10


Eigenvalues: [ 6.  1. -1.],
Eigenvectors: [[ 4.61566331e-01  8.94427191e-01  5.18104078e-17]
 [ 5.93442426e-01 -4.47213595e-01 -4.47213595e-01]
 [ 6.59380473e-01  3.01974496e-16  8.94427191e-01]]
```



Question 4: Face Recognition with Eigenfaces (30 points + 10 bonus points)

Goal: Perform face recognition on the *Labeled Faces in the Wild* dataset using PyTorch.

Dataset Information: *Labeled Faces in the Wild* dataset consists of face photographs

designed for studying the problem of unconstrained face recognition. The original dataset contains more than 13,000 images of faces collected from the web.

Tasks:

- First, perform Principal Component Analysis (PCA) on the image dataset.
- Using PCA, extract the Top k principal components (*eigenvalues*).
- Reconstruction of faces from these *eigenvalues* will give us the *eigen-faces* which are the most representative features of most of the images in the dataset.
- **BONUS**: Finally, train a simple PyTorch Neural Network model on the modified image dataset. This trained model will be used for prediction and evaluation on a test set.

Note:

- Run all the cells in order.
- **Do not edit** the cells marked with `!!DO NOT EDIT!!`
- Only **add your code** to cells marked with `!!!! YOUR CODE HERE !!!!`
- Do not change variable names, and use the names which are suggested.

In [3]:
```
# !!DO NOT EDIT!!
# loading the dataset directly from the scikit-learn library (can take about 3-5 mi
import numpy as np
from sklearn.datasets import fetch_lfw_people
dataset = fetch_lfw_people(min_faces_per_person=80)

# each 2D image is of size 62 x 47 pixels, represented by a 2D array.
# the value of each pixel is a real value from 0 to 255.
count, height, width = dataset.images.shape
print('The dataset type is:',type(dataset.images))
print('The number of images in the dataset:',count)
print('The height of each image:',height)
print('The width of each image:',width)

# sklearn also gives us a flattened version of the images which is a vector of size
# we can directly use that for our exercise
print('The shape of data is:',dataset.data.shape)
```

```
The dataset type is: <class 'numpy.ndarray'>
The number of images in the dataset: 1140
The height of each image: 62
The width of each image: 47
The shape of data is: (1140, 2914)
```

For optimum performance, we have only considered people who have more than 80 images. This restriction notably reduces the size of the dataset.
Now let us look at the labels of the people present in the dataset

In [4]:
```
# !!DO NOT EDIT!!
# create target label - target name pairs
```

```
targets = [(x,y) for x,y in zip(range(len(np.unique(dataset.target))), dataset.targ
print('The target labels and names are:\n', targets)
```

```
The target labels and names are:
 [(0, np.str_('Colin Powell')), (1, np.str_('Donald Rumsfeld')), (2, np.str_('George
W Bush')), (3, np.str_('Gerhard Schroeder')), (4, np.str_('Tony Blair'))]
```

(a) Preprocessing:

Using the `train_test_split` API from `sklearn`, split the data into train and test dataset in the ratio 3:1. Use `random_state=42`.

For better performance, normalize the features which can have different ranges with huge values. (As all our features here are in the range [0,255], it is not explicitly needed here. However, it is a good exercise.)

Use the `StandardScaler` class from sklearn and use that to normalize `X_train` and `X_test`. Validate and show your result by printing the first 5 features of 5 images of `X_train` (This result can vary from pc to pc). (10 points)

In [5]:
```
# !!DO NOT EDIT!!
X = dataset.data
y = dataset.target
```

In [6]:
```
#######
# !!!! YOUR CODE HERE !!!!
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# first 5 features of 5 images of `X_train`
print(X_train_scaled[:5, :5])
# output variable names -  X_train, X_test, y_train, y_test
#######
```

```
[[-1.0548956  -1.1314403  -1.1468822  -0.85341436 -0.7309732 ]
 [ 2.6098442   2.4664047   1.6027023   0.7667316   0.23976761]
 [-1.0626272  -1.0687327  -1.1071484  -1.1075549   1.3457758 ]
 [-0.1735025  -0.2221809  -0.7018628  -1.3775792  -1.3436539 ]
 [ 0.05844303  0.00513394 -0.03433381 -0.20217922 -0.0466805 ]]
```

(b) Dimensionality reduction :

Use the `PCA` API from `sklearn` to extract the top 100 principal components of the image

matrix and fit it on the training dataset.

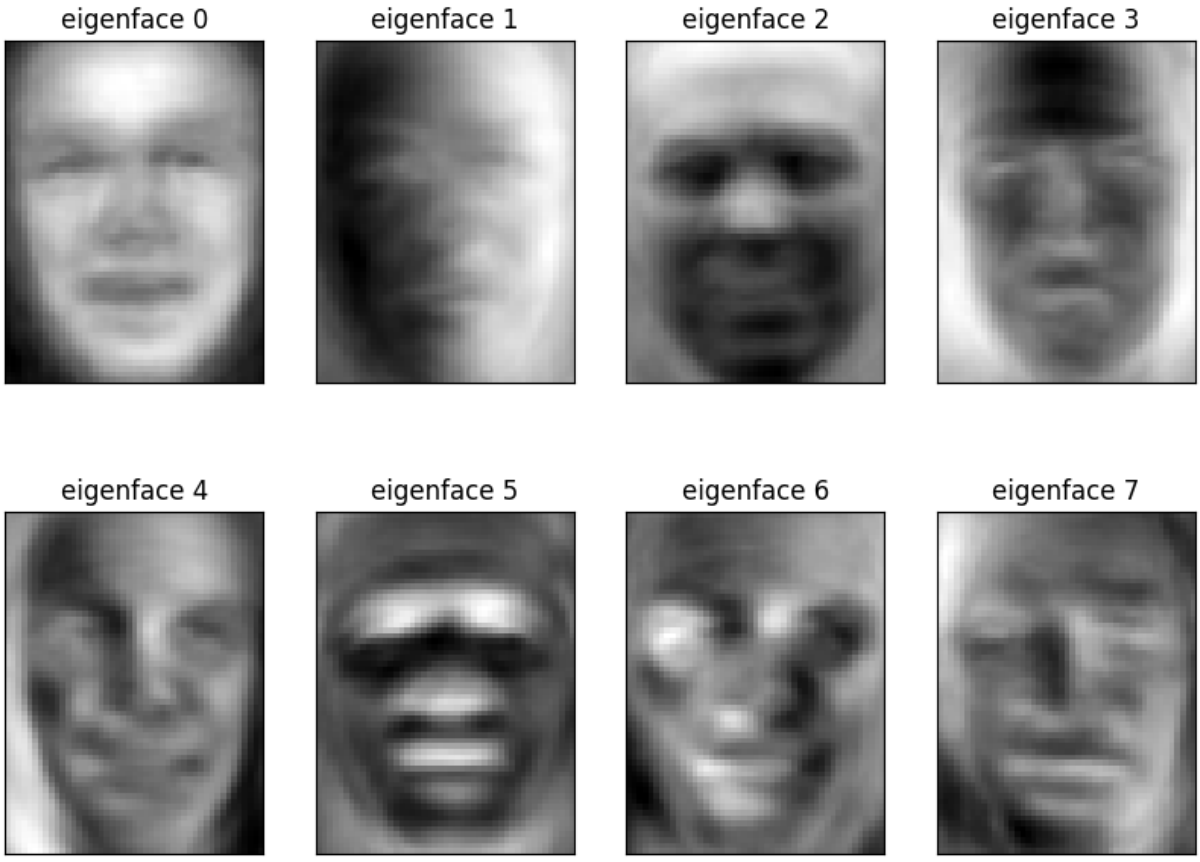
Visualize some of the top few components as an image (eigenfaces). (10 points)

In [7]:
```
#######
# !!!! YOUR CODE HERE !!!!
# initialize PCA API from sklearn with n_components. Also set svd_solver="randomize

from sklearn.decomposition import PCA

n_components = 100
pca_transform = PCA(n_components=n_components, svd_solver='randomized', whiten=True
pca = pca_transform.fit(X_train_scaled)
# output variable name -  pca
#######
```

Now we will plot the most representative eigenfaces:

In [8]:
```
# !!DO NOT EDIT!!
# Helper function to plot
import matplotlib.pyplot as plt
def plot_gallery(images, titles, height, width, n_row=2, n_col=4):
    plt.figure(figsize=(2* n_col, 3 * n_row))
    plt.subplots_adjust(bottom=0, left=0.01, right=0.99, top=0.90, hspace=0.35)
    for i in range(n_row * n_col):
        plt.subplot(n_row, n_col, i + 1)
        plt.imshow(images[i].reshape((height, width)), cmap=plt.cm.gray)
        plt.title(titles[i], size=12)
        plt.xticks(())
        plt.yticks(())
```

In [9]:
```
# !!DO NOT EDIT!!
# get the 100 eigen faces and reshape them to original image size which is 62 x 47
eigenfaces = pca.components_.reshape((n_components, height, width))

# plot the top 8 eigenfaces
eigenface_titles = ["eigenface %d" % i for i in range(eigenfaces.shape[0])]
plot_gallery(eigenfaces, eigenface_titles, height, width)

plt.show()
```



(c) Face reconstruction:

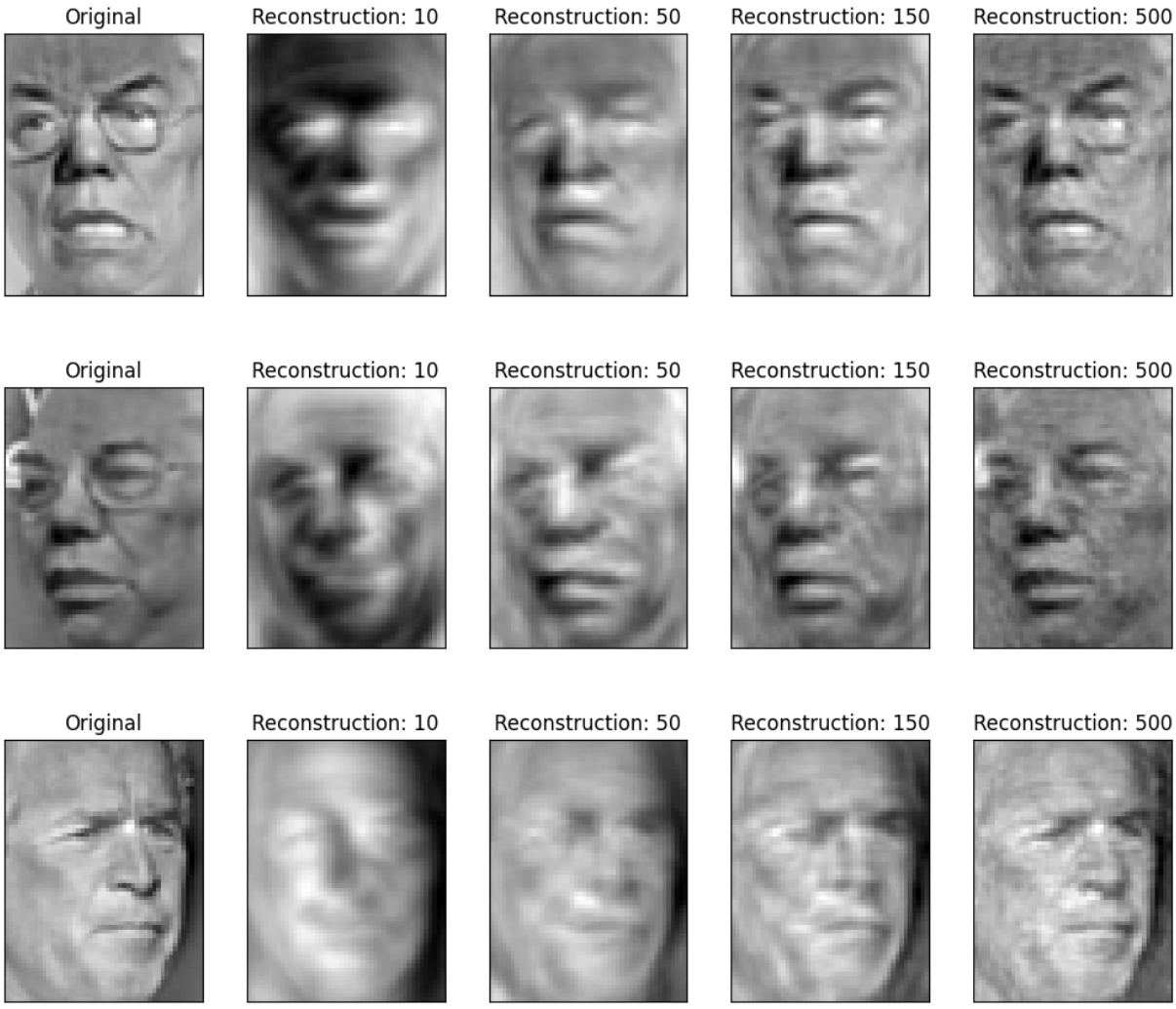
Reconstruct an image from its point projected on the principal component basis.

Project the first three faces on the eigenvector basis using PCA models trained with varying number of principal components. Using the projected points, reconstruct the faces, and visualize the images.

Your final output should be a (3×5) image matrix, where the rows are the data points, and the columns correspond to original image and reconstructed image for n_components $= [10, 100, 150, 500]$. (10 points)

In [10]:
```
#######
# !!!! YOUR CODE HERE !!!!
n_components_lst = [10, 50, 150, 500]

reconstructed_faces = []
for n_components in n_components_lst:
    pca_transform = PCA(n_components=n_components, svd_solver='randomized', whiten=
    pca = pca_transform.fit(X_train_scaled)
    projection = pca.transform(X_test_scaled[:3])
    face = pca.inverse_transform(projection)
    reconstructed_faces.append(face)

output = []
```

```
for i in range(3): # each face is a row, 3x1 output
    output.append(X_test_scaled[i]) # IndexError: index 3 is out of bounds for axis
    for j in range(len(n_components_lst)): # now to make 4 more columns
        output.append(reconstructed_faces[j][i])

titles = []
for i in range(3): # each face is a row, 3x1 columns
    titles.append('Original') # IndexError: index 3 is out of bounds for axis 0 wit
    for component in n_components_lst: # now to make 4 more columns
        titles.append(f'Reconstruction: {component}')

# plot_gallery() wants ALL the args
plot_gallery(output, titles, height, width, n_row=3, n_col=5)
plt.show()
#######
```



(d) Prediction (Bonus):

Train a neural network classifier in **PyTorch** on the transformed dataset. Complete each of the steps below.

Note: For PyTorch reference see documentation. (10 points)

In [11]:
```
# !!DO NOT EDIT!!
# define imports here
import torch
import torch.nn as nn
```

In [12]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Before training, we need to transform the training and test dataset to reduced forms (100 dimensions) using the `pca` function defined in (b).

Move the train and test dataset to torch tensors in order to work with pytorch.

In [13]:
```
#######
# !!!! YOUR CODE HERE !!!!
# 1. project X_train and X_test on orthonormal basis using the PCA API initialized
n_components = 100
pca_transform = PCA(n_components=n_components, svd_solver='randomized', whiten=True
pca = pca_transform.fit(X_train_scaled)
X_train_pca = pca.transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# 2. now convert X_train_pca, X_test_pca, y_train and y_test to torch.tensor. For y
X_train_pca_torch = torch.tensor(X_train_pca).to(device)
X_test_pca_torch = torch.tensor(X_test_pca).to(device)
y_train_torch = torch.tensor(y_train, dtype=torch.long).to(device)
y_test_torch = torch.tensor(y_test, dtype=torch.long).to(device)

# output variable names -  X_train_pca_torch, X_test_pca_torch, y_train_torch, y_te
#######
```

In [14]:
```
#######
# !!!! YOUR CODE HERE !!!!
# 3. We will implement a simple multilayer perceptron (MLP) in pytorch with one hid
# Using this neural network model, we will train on the transformed dataset.
class MLP(torch.nn.Module):
  def __init__(self):
    super(MLP, self).__init__()
    # Initalize various layers of MLP as instructed below
    # DO: initialze two linear layers: 100 -> 1024  and 1024-> 5
    self.linearlayerone = nn.Linear(100, 1024)
    self.linearlayertwo = nn.Linear(1024, 5)
    # DO: initialize relu activation function
    self.relu = nn.ReLU()
    # DO: initialize LogSoftmax
    self.logsoftmax = nn.LogSoftmax()
  def forward(self, x):
    # DO: define the feedforward algorithm of the model and return the final output
    x = self.linearlayerone(x)
    x = self.relu(x)
    x = self.linearlayertwo(x)
    x = self.logsoftmax(x)
    return x
```

```
#######
```

In [15]:
```
#######
# !!!! YOUR CODE HERE !!!!
# 4. create an instance of the MLP class here
model = MLP().to(device)
# 5. define loss (use negative log likelihood loss: torch.nn.NLLLoss)
criterion = nn.NLLLoss().to(device)
# 6. define optimizer (use torch.optim.SGD (Stochastic Gradient Descent)).
# Set learning rate to 1e-1 and also set model parameters
optimizer = torch.optim.SGD(model.parameters(), lr=1e-1)
#######

# !!DO NOT EDIT!!
# 7. train the classifier on the PCA-transformed training data for 500 epochs
# This part is already implemented.
# Go through each step carefully and understand what it does.
for epoch in range(501):
  # reset gradients
  optimizer.zero_grad()

  # predict
  output=model(X_train_pca_torch)

  # calculate loss
  loss=criterion(output, y_train_torch)

  # backpropagate loss
  loss.backward()

  # performs a single gradient update step
  optimizer.step()

  if epoch%50==0:
    print('Epoch: {}, Loss: {:.3f}'.format(epoch, loss.item()))
```

```
d:\DSC 210\.venv\Lib\site-packages\torch\nn\modules\module.py:1775: UserWarning: Implicit dimension choice for log_softmax has been deprecated. Change the call to include dim=X as an argument.
  return self._call_impl(*args, **kwargs)
Epoch: 0, Loss: 1.605
Epoch: 50, Loss: 0.425
Epoch: 100, Loss: 0.215
Epoch: 150, Loss: 0.131
Epoch: 200, Loss: 0.088
Epoch: 250, Loss: 0.064
Epoch: 300, Loss: 0.049
Epoch: 350, Loss: 0.039
Epoch: 400, Loss: 0.032
Epoch: 450, Loss: 0.027
Epoch: 500, Loss: 0.023
```

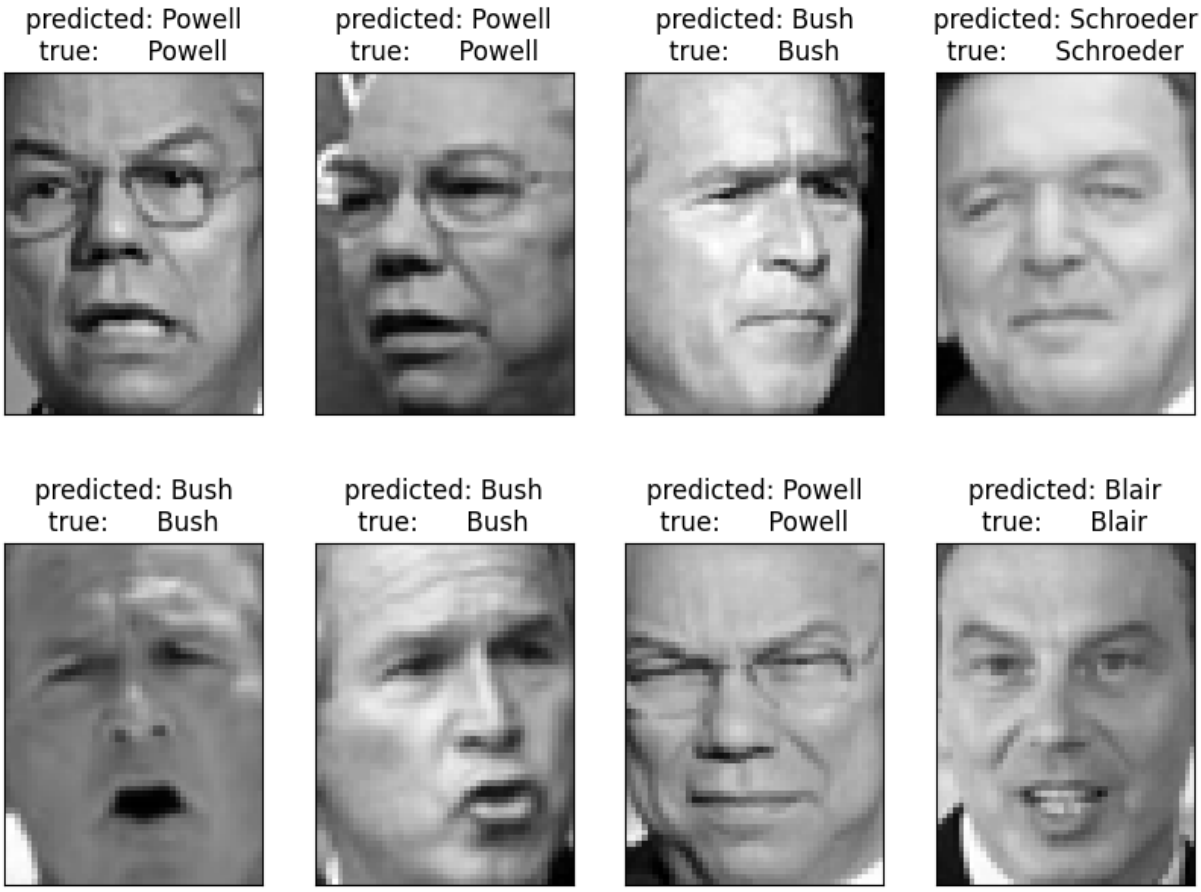
In [16]:
```
# !!DO NOT EDIT!!
# predict on test data
predictions = model(X_test_pca_torch) # gives softmax logits
```

```
y_pred = torch.argmax(predictions, dim=1).numpy() # get the labels from prdictions:
```

In [17]:

```
# !!DO NOT EDIT!!
# here, we will print the multi-label classification report: precision, recall, f1-
from sklearn.metrics import classification_report
target_names=[y for x,y in targets]
print(classification_report(y_test, y_pred, target_names=target_names))

# let us validate some of the predictions by plotting images
# display some of the results
def title(y_pred, y_test, target_names, i):
    pred_name = target_names[y_pred[i]].rsplit(" ", 1)[-1]
    true_name = target_names[y_test[i]].rsplit(" ", 1)[-1]
    return "predicted: %s\ntrue:      %s" % (pred_name, true_name)

prediction_titles = [
    title(y_pred, y_test, target_names, i) for i in range(y_pred.shape[0])
]

plot_gallery(X_test, prediction_titles, height, width)
```

```
                   precision    recall  f1-score   support

     Colin Powell       0.93      0.86      0.89        64
  Donald Rumsfeld       0.86      0.75      0.80        32
    George W Bush       0.90      0.96      0.93       127
Gerhard Schroeder       0.86      0.83      0.84        29
       Tony Blair       0.86      0.91      0.88        33

         accuracy                           0.89       285
        macro avg       0.88      0.86      0.87       285
     weighted avg       0.89      0.89      0.89       285
```


predicted: Powell
true: Powell
predicted: Powell
true: Powell
predicted: Bush
true: Bush
predicted: Schroeder
true: Schroeder
predicted: Bush
true: Bush
predicted: Bush
true: Bush
predicted: Powell
true: Powell
predicted: Blair
true: Blair